Деплой Foodies на сервер HestiaCP

Автоматический деплой из GitHub (jamperkostya/foodies-academy) через GitHub Actions + SSH/rsync

Исходные условия

- **Hestia не работает с Docker.** Это обычная панель с nginx, Apache и PHP-FPM. Поэтому `docker-compose.yml` на сервер не переносится, каждое приложение деплоится своим способом.
- `astro-app` **собирается в статику:** в `astro.config.mjs` нет адаптера, `npm run build` кладёт готовый сайт в `dist/`. На сервер нужно отправить только эту папку, Node.js там не нужен.
- Схема: при push в **main** GitHub Actions собирает сайт и копирует его на сервер по SSH (rsync).
- Порт Astro в локальном `docker-compose.yml` сейчас `8082`. На деплой через Hestia это не влияет, но если compose запускать на сервере, порт `80` будет конфликтовать с nginx Hestia.

1. Что сделать в Hestia

1. **Web → Add Web Domain.** Создайте домен, например `foodies.example.com`, и включите SSL (Let's Encrypt).
2. **Users → ваш пользователь → Edit.** В поле **SSH Access** поставьте `bash`. По умолчанию там `nologin`, и rsync работать не будет.
3. Сгенерируйте отдельный ключ только для деплоя. Команда локально:

```
ssh-keygen -t ed25519 -f ~/.ssh/foodies_deploy -N "" -C "github-deploy"
```

   Содержимое `~/.ssh/foodies_deploy.pub` добавьте в Hestia: **User → Manage SSH Keys**. Можно и вручную, в `~/.ssh/authorized_keys` на сервере.

4. Проверьте подключение:

```
ssh -i ~/.ssh/foodies_deploy USER@SERVER_IP "ls ~/web/foodies.example.com/public_html"
```

2. Секреты в GitHub

В репозитории откройте **Settings → Secrets and variables → Actions** и добавьте:

Секрет	Значение
`SSH_HOST`	IP или хост сервера
`SSH_USER`	пользователь Hestia
`SSH_PORT`	обычно 22
`SSH_KEY`	содержимое ПРИВАТНОГО ключа ~/.ssh/foodies_deploy
`DEPLOY_PATH`	/home/USER/web/foodies.example.com/public_html

3. Workflow: `.github/workflows/deploy.yml`

```
name: Deploy Astro to Hestia

on:
  push:
    branches: [main]
    paths: ['astro-app/**', '.github/workflows/deploy.yml']
  workflow_dispatch:

concurrency: deploy-production

jobs:
  deploy:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: astro-app
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: astro-app/package-lock.json

      - run: npm ci
      - run: npm test
      - run: npm run build

      - name: Setup SSH
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.SSH_KEY }}" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -p ${{ secrets.SSH_PORT }} ${{ secrets.SSH_HOST }} >> ~/.ssh/known_hosts

      - name: Rsync dist to server
        run: |
          rsync -az --delete \
            -e "ssh -p ${{ secrets.SSH_PORT }}" \
            dist/ ${{ secrets.SSH_USER }}@${{ secrets.SSH_HOST }}:${{ secrets.DEPLOY_PATH }}/
```

Внимание: с флагом `--delete` rsync удаляет из `public_html` всё, чего нет в `dist/`, в том числе стандартную заглушку Hestia. Так и задумано, но в эту папку нельзя класть ничего вручную: при следующем деплое оно пропадёт.

4. Laravel и WordPress (если их тоже нужно выкладывать)

Для каждого создайте в Hestia отдельный домен или поддомен и базу данных (**DB → Add Database**). Логины и пароли из `docker-compose.yml` для продакшена не используйте.

- **Laravel:** в настройках домена поставьте proxy/backend template с корнем в `public/`. Деплой делается через SSH на сервере:

```
git pull && composer install --no-dev -o \
  && php artisan migrate --force && php artisan config:cache
```

  Файл `.env` создайте на сервере руками, в git он не хранится.

- **WordPress:** ядро поставьте через Hestia (**Quick Install App → WordPress**). Из git деплойте только `wp-content/themes` и `wp-content/plugins` через rsync, так же как Astro. Папку `uploads` не трогайте, флаг `--delete` к ней не применяйте.